

STAT 184 - Homework 4: Joining Tables

Due Monday, June 15 at 11:59 PM on Canvas

Your Name Here

AI Disclosure

- ☐ No AI used
- ☐ Syntax / Debugging (e.g., finding a missing comma, fixing an error code)
- ☐ Conceptual explanation (e.g., asking how `left_join()` works)
- ☐ Code Generation (e.g., asking AI to write a join pipeline from scratch)

Briefly describe your use (1-2 sentences):

[Type your explanation here]

Setup

Run the chunk below to load the packages. `nycflights13` should already be installed from HW 3. If not, run `install.packages("nycflights13")` in the **Console** once.

```
library(tidyverse)
library(nycflights13)
```

This homework uses the five related tables in `nycflights13`: `flights`, `airlines`, `airports`, `planes`, and `weather`. Take a moment to look at `flights`:

```
glimpse(flights)
```

Rows: 336,776

Columns: 19

```
$ year      <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
$ month     <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ day       <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ dep_time  <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ~
$ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ~
$ dep_delay <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -
1~
$ arr_time  <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849,~
$ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851,~
$ arr_delay <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -
1~
$ carrier   <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "~
$ flight    <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4~
$ tailnum   <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394~
$ origin    <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA",~
$ dest      <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD",~
$ air_time  <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1~
$ distance  <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ~
$ hour      <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6~
$ minute    <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0~
$ time_hour <dtm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-
01 0~
```

Throughout this homework, the key relationship to keep in mind: `flights` is the central table. The other four tables each add detail about one piece of a flight: the airline, the airports, the plane, or the weather conditions.

Question 1 – left_join basics

`left_join()` keeps every row from the left table and attaches matching columns from the right. If there's no match, the new columns get `NA`.

Part (a): add airline names

The `carrier` column in `flights` holds two-letter codes. Use `left_join()` to attach the full airline name from `airlines`, then build a summary: **for each airline, how many flights**

departed, and what was the average departure delay? Include only carriers with at least 1,000 flights, arrange by average delay (highest first), and make sure to output the result.

```
# Write your R code here
```

Which airline had the worst average departure delay?

[Write your answer here.]

Part (b): add plane details

Use `left_join()` to attach `manufacturer` and `seats` from `planes` to a filtered version of `flights`. Narrow `flights` down to just `month`, `day`, `carrier`, `tailnum`, `origin`, and `dest` first, then join. Use `head()` to output the first six rows.

```
# Write your R code here
```

Part (c): when there's no match

Not every tail number in `flights` has a record in `planes`. After joining in Part (b), filter the result to only rows where `manufacturer` is NA. How many such flights are there?

```
# Write your R code here
```

What does it mean that `manufacturer` is NA here?

[Write your answer here.]

Question 2 – `join_by()` with different column names

Sometimes the key column has a different name in the two tables. `join_by(a == b)` says: match rows where column `a` in the left table equals column `b` in the right table.

The `airports` table has a primary key called `faa`. In `flights`, both `origin` and `dest` refer to airport codes, but they're named differently from `faa`.

Part (a): destination airport names

Join `airports` onto `flights` using `dest` as the key. Select only `month`, `day`, `origin`, `dest`, and `name` (the destination airport's full name) from the result. Show the first six rows.

```
# Write your R code here
```

Part (b): most popular destinations

Using the join from Part (a), find the **10 most common destination airports by number of flights**. Your result should show the airport code, the full airport name, and the flight count. Arrange by count descending.

```
# Write your R code here
```

Part (c): destination coordinates

Join the latitude (`lat`) and longitude (`lon`) from `airports` onto `flights` using `dest`. Then compute the **average arrival delay by destination airport**, keeping only airports with at least 100 flights. Join the airport name in as well so the table is readable. Arrange by average delay descending and show the top 10.

```
# Write your R code here
```

Do the airports with the worst average arrival delays share anything in common (region, size, etc.)?

[Write your answer here.]

Question 3 – `anti_join` and `semi_join`

Filtering joins don't add columns; they filter rows based on whether a match exists in another table.

- `anti_join(x, y)`: keeps rows in `x` with **no** match in `y`
- `semi_join(x, y)`: keeps rows in `x` **with** a match in `y`

Part (a): flights with no plane record

Use `anti_join()` to find all flights whose `tailnum` does not appear in `planes`. How many such flights are there? Then use `count()` on `carrier` to see which airlines account for most of the missing records.

```
# Write your R code here
```

Which carriers account for most of the unmatched tail numbers? What might explain this?

[Write your answer here.]

Part (b): filtering airports by flight activity

The `airports` table has 1,458 airports, but most of them never appear as a destination in `flights`. Use `semi_join()` to keep only the airports that appear as a `dest` in `flights`. How many airports remain?

```
# Write your R code here
```

Part (c): thinking about the joins

Answer each of the following:

1. Why would you use `anti_join()` instead of just `filter()` to find unmatched rows?
2. If `anti_join(flights, planes, join_by(tailnum))` gives you rows, what does that tell you about the data?

[Write your answer here.]

Question 4 – joins in a pipeline

Joins fit naturally into the same pipelines you've been building all semester. This question asks you to chain joins with the wrangling verbs from weeks 3 and 4.

Part (a): average delay by airline name

The goal: a table showing **average arrival delay by full airline name**, for airlines with at least 500 flights, arranged from worst to best. You'll need to join `airlines` to get the name, group by it, summarise, filter, and arrange.

```
# Write your R code here
```

Part (b): delay by plane age

The `planes` table has a `year` column giving the year each plane was manufactured. Join `planes` onto `flights`, compute the plane's age as `2013 - year` (since `flights` is from 2013), then group by age and compute average departure delay. Show only planes where age is known (not NA) and where the age group has at least 100 flights.

Make a scatter plot with plane age on the x-axis and average departure delay on the y-axis.

```
# Write your R code here
```

Is there a visible relationship between plane age and departure delay in this data?

[Write your answer here.]

Code appendix

```
library(tidyverse)
library(nycflights13)
glimpse(flights)
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
# Write your R code here
```